CS & IT ENGINEERING



Algorithms

Miscellaneous

Lecture No.- 004



By- Aditya sir

Recap of Previous Lecture



- Topic
- Topic

OBST
↳ Part 1

Topics to be Covered



Topic

Topic

OBST → part 2 (DP)

Backtracking & Branch Bound

Tips & Tricks

ABOUT ME

Hello, I'm Aditya.

1. Represented college as the first Google DSC Ambassador.
2. The only student from the batch to secure an internship at Amazon.
3. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis8. Completed my Masters with an overall GPA of 9.36/108. Joined Dream11 as a Data Scientist.9. Have mentored around 10k working professions in field of Data Science and Analytics10. Have been mentoring GATE aspirants to secure a great rank in limited time.11. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



By- Aditya sir

Telegram



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : OBST

OBST : Type 1 Question :

$A_{1 \to n}$

$P_{1 \to n}$

$q_{0 \to n}$



Topic : OBST

#Q. How many different BSTs for a given Binary Tree with n nodes?

$$\frac{1}{(n+1)} * {}^{2n}C_n$$



Topic : OBST

Solution:

- Catalan Number = $\frac{1}{(n+1)} \times {}^{2n}C_n$
- Cost (T) = Cost (successful searches) + Cost (unsuccessful searches)

$$= \sum_{i=1}^{n} P_i * \text{level}(a_i) + \sum_{i=0}^{n} q_i * (\text{level}(E_i) - 1)$$

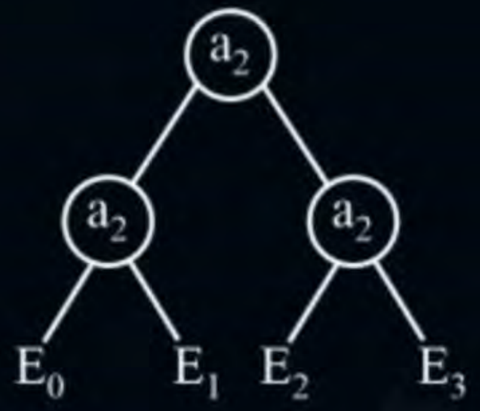


Topic : OBST

n-identifiers:

$a_1 \rightarrow a_n$

$E_0 \rightarrow E_n$



Conclusion:

- The cost of BST is dependent on both :
  - the height (depth) as well as
  - the probabilities (p_i & q_i) → frequencies of identifiers



Topic : OBST

Type-2: Construction of Optimal Cost BST (DP based)

Input: $A = [a_1, a_2, \ldots, q_n]$

$P = [p_1, p_2, \ldots, p_n]$

$Q = [q_0, q_1, \ldots, q_n]$

- In a BST with n nodes:

  $n \rightarrow$ identifiers (a_i)

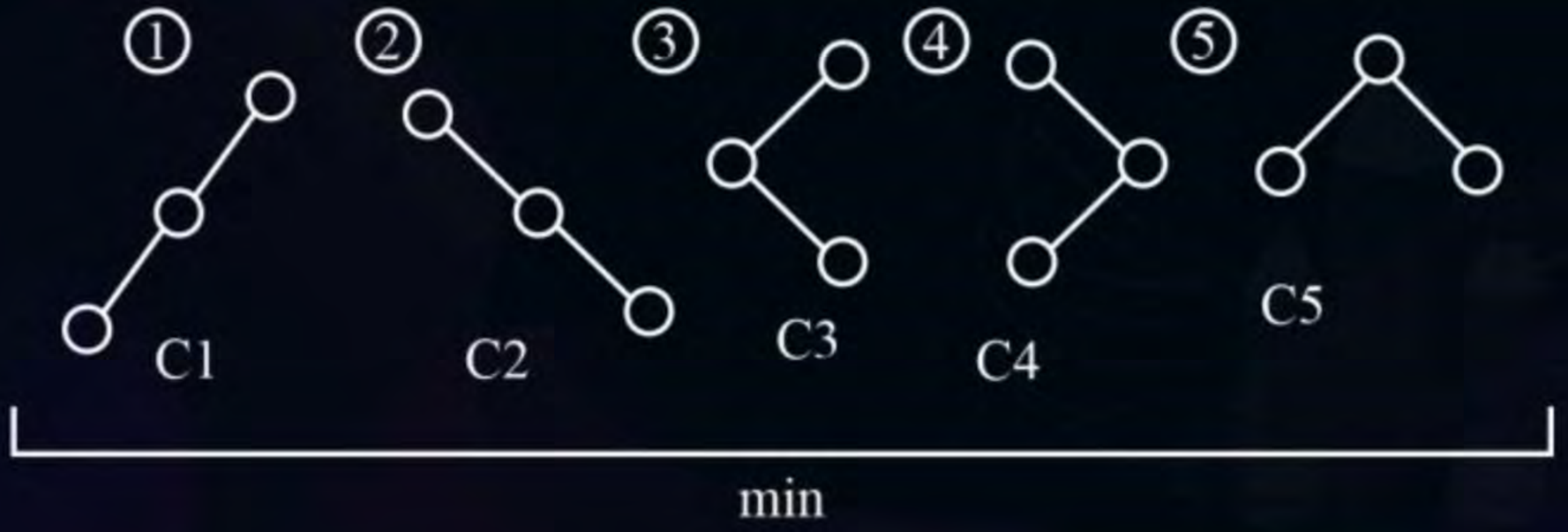
  $(n + 1) \rightarrow$ null links (E_i)



Topic : OBST

E.g.: n = 3, no. of different BSTs. = 5

Process of Enumeration / Brute-Force:





Topic : OBST

No. of nodes	No. of diff BSTs
n = 3	5
n = 4	14

n = 4

$$= \frac{1}{(4+1)} \times {}^{8}C_{4}$$

$$= \frac{1}{5} \times \frac{8!}{4! \times 4!}$$

$$= \frac{1}{5} \times \frac{8 \times 7 \times 6 \times 5 \times 4!}{4! \times 4!}$$

$$= \frac{8 \times 7 \times 6}{4 \times 3 \times 2}$$

$$= 2 \times 7 = 14$$



Topic : OBST

Imagine : n = 300

$$\text{No. of BSTs} = \frac{1}{(301)} \times {}^{600}C_{300}$$

- Hence, Brute Force Approach is NOT feasible for large n. (As no. of BSTs increase exponentially with increase in n)
- ~~Some~~ need a better approach/optimal.
- Dynamic Programming (DP)



Topic : OBST

DP based Approach for Construction of OBST:

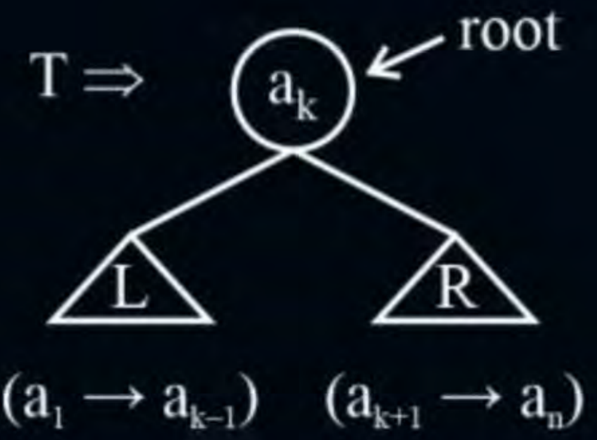
$A = [a_1, a_2, \ldots, a_n]$

$\text{Cost}(T) = \text{Cost}(a_k) + \text{Cost}(L) + \text{Cost}(R)$

$a_k \rightarrow$ root node

For root, level = 1

Probability = P_K



$\text{Cost}(a_k) = \text{Prob}(a_k) \times \text{level}(a_k)$

$= P_k \times 1$

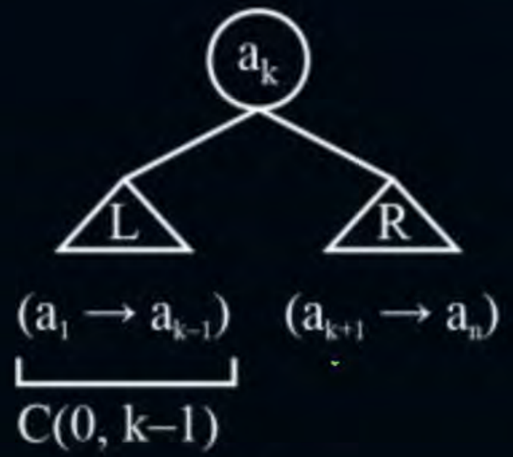
$= P_K$



Topic : OBST

$$\text{Cost}(T) = \text{Cost}(a_k) + \text{Cost}(L) + \text{Cost}(R)$$

$$\text{Cost}(T) = P_k + \text{Cost}(L) + \text{Cost}(R)$$



Let C(0, n) represent cost of BST $T_{1 \rightarrow n}$

$$\text{Cost}(T_{1 \rightarrow n}) = C(0, n) = \{P_K + C(0, k-1) + C(k, n)\} + W(0, n); \qquad 0 < K \leq n$$

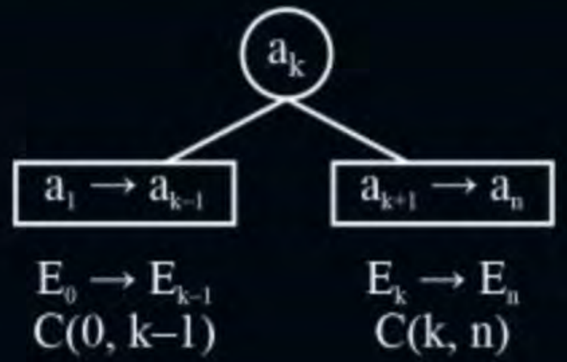
NOTE: W(0, n) is added to the cost of LST and RST to balance it with respect to the root.



Topic : OBST

Explanation:

- $T_{1 \to n} = C(0, n) = \{P_k + \text{Cost}(L) + \text{Cost}(R)\} + W(0, n)$





Topic : OBST

Need for W(0, n) → Explanation (~~intention~~ intuition):

Overall level of a_3?

⇒ 2

a_2 is root of LST

→ Level of a_3 within LST is 1





Topic : OBST

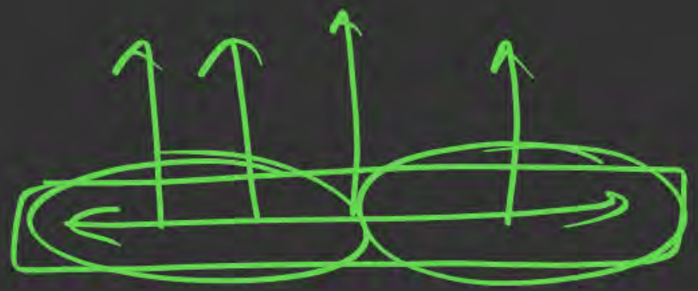
Remember this

Generalizing for any subtree a_{ij} (root is a_k):

$$C(i, j) = \min \{P_K + C(i, k-1) + C(k, j) + W(i, j)\}; \; i < k \leq j \qquad \ldots(i)$$

where k is such that, having a_k as root will minimize equation (i)

Base Condition, $C(i, j) = 0$ (0 nodes)





Topic : OBST

NOTE:

Time Complexity of Previous DP based approach.

TC = $O(n^3)$ {DP}

Brute Force → DP based

Expo

$O(n^3)$ Polynomial

$\frac{1}{(n+1)} {}^{2n}C_n$



Topic : OBST

Check for overlapping Sub-problems:



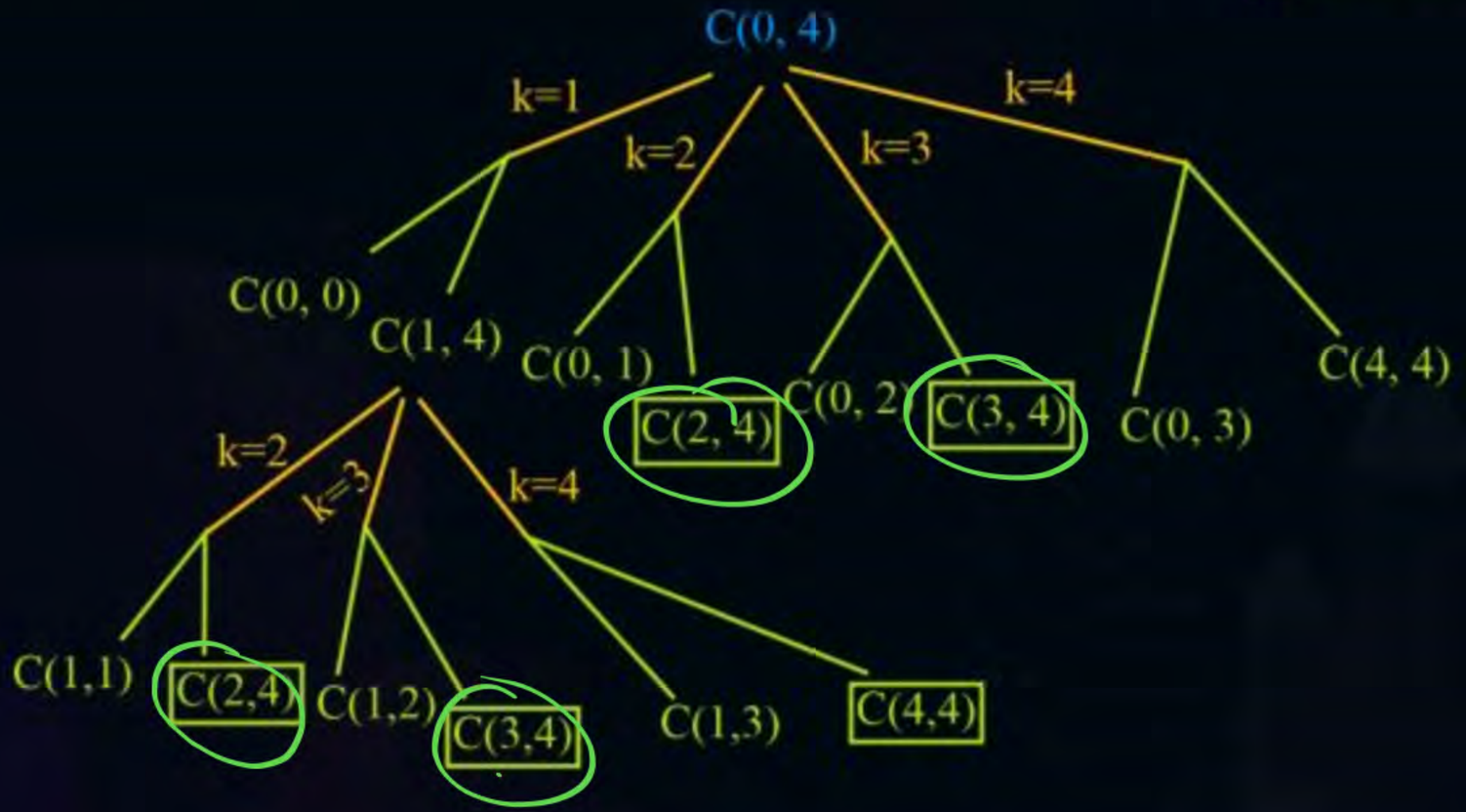
E.g.: $A = [a_1, a_2, a_3, a_4]$



Topic : OBST



$i < k \leq j$



Overlapping Sub-problems exist



Topic : Back Tracking

- Represents one of the most general techniques.
- Many problems which deal with searching for a set of solutions or which ask for an optimal solution satisfying some constraints can be solved using the backtracking formulation.
- In order to apply the backtrack method, the desired solution must be expressible as an n-tuple (x_1, x) where the x_i are chosen from some finite set S_i. Often the problem to be solved calls for finding one vector which maximizes (or minimizes or satisfies) a criterion function $P(x_1, ..., x_n)$. Sometimes it seeks all such vectors which satisfy P.
- For example



Topic : Back Tracking

E.g.: n-Queens Problem:

Decision

- Given n Queens $(q_1, q_2, \ldots, q_n)$

  $X = [x_1 \rightarrow x_n] \rightarrow$ feasible solution

  $X = [x_1, x_2, \ldots, x_n]$

  X_i = Position of Queen Q_i in the ith row of n × n matrix (chess board)

  $1 \leq X_i \leq n$



Topic : Back Tracking

X = [1, 2, 3, 4]

X = [1, 2, 3, 4]

⋮

⋮

Solution Space

State Space
(Backtracking)



Topic : Back Tracking

E.g.: n = 4 (q_1, q_2, q_3, q_4)

X = [1, 3, _, _]

X = [1, _, _, _]

X = [2, 4, 1, 3] → feasible solution

[_, _, _, _]

	1	1	1	1
q_1				
q_2				
q_3				
q_4				



Topic : Back Tracking

State Space Tree:

(1) Fixed Size Tuple:

E.g.: n-Queens

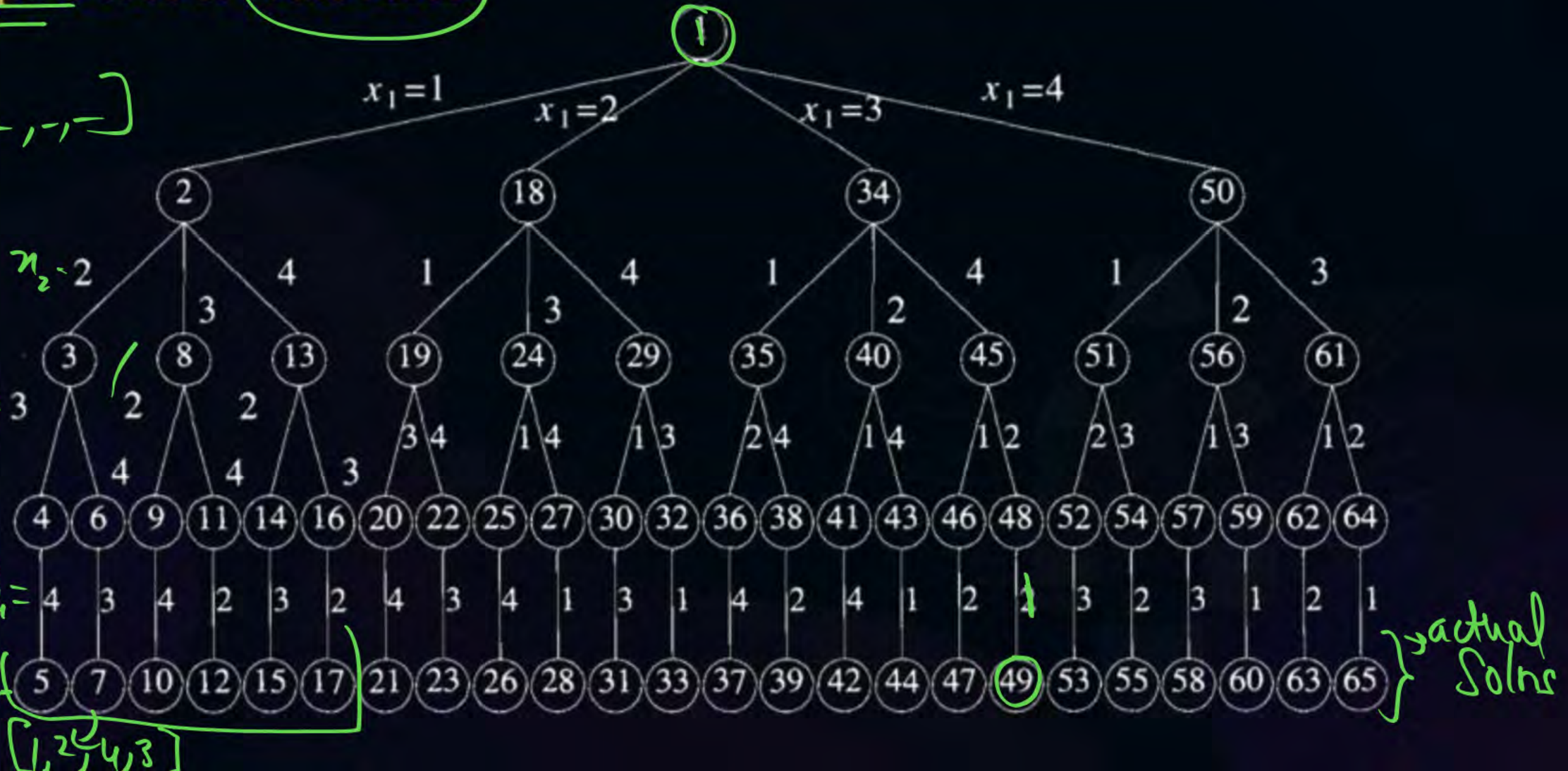
(2) Variable Size Tuple:

E.g.: Sum of Subsets (SOS)



Topic : Back Tracking

State Space Tree of Brute Force :





Topic : Back Tracking

X = [1, 2, 3, 4]

X = [1, 2, 3, 4]

⋮

⋮

n!

Total branches = 6 × 4 = 24

n! = Size of solution space / State Space

n = 4

4! = 24



Topic : Back Tracking

V. Imp

Important Points:

(1) Backtracking uses DFS.

(2) DFS is used to find the feasible solution among all the solutions in the state space.

(3) It applies a bounding function to terminate/stop those nodes that can never lead to an optimal/feasible solution.



Topic : Back Tracking

Backtracking:

- ~~DGS~~ along with some Bounding Function.

DFS



Topic : Back Tracking

Potential Question:

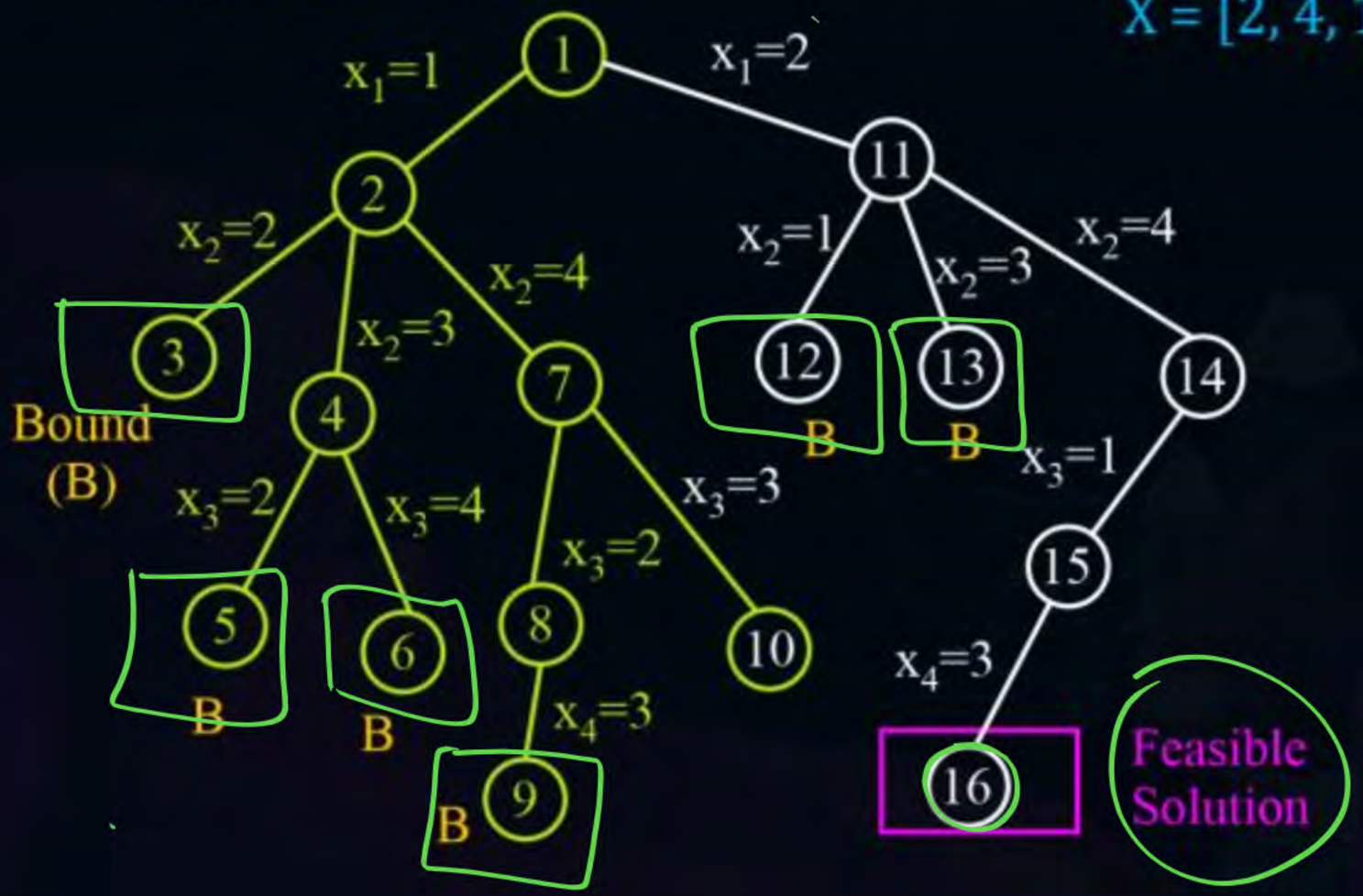
- Portion of the State Space Tree that gets generated due to backtracking process for n = 4 Queens Problem.
- To get 1st feasible solution: 16 states



Topic : Back Tracking

State Space Tree using Back-tracking:

X = [2, 4, 1, 3]



Topic : Back Tracking

	1	1	1	1
q_1		✓		
q_2				✓
q_3	✓			
q_4			✓	





Topic : Back Tracking

Example of Variable Sized Tuple Problem and Solution using Backtracking

(State Space Tree):

Sum of Subset (SOS):

- A[n] : n elements A_1, …, A_n ; M = Target sum



Topic : Back Tracking

E.g.:

n = 6, M = 50 (Target Sum)

A = [20, 40, 50, 10, 30, 5]

1 2 3 4 5 6

indices

S1 = {3}

S2 = {2, 4}

S3 = {1, 5}

feasible solution

Variable sized tuple as the feasible solutions are not at the same level (size) in the state space Tree.



Topic : Back Tracking

Size
~~Sum~~ of the State Space for SOS problem with 'n' elements:

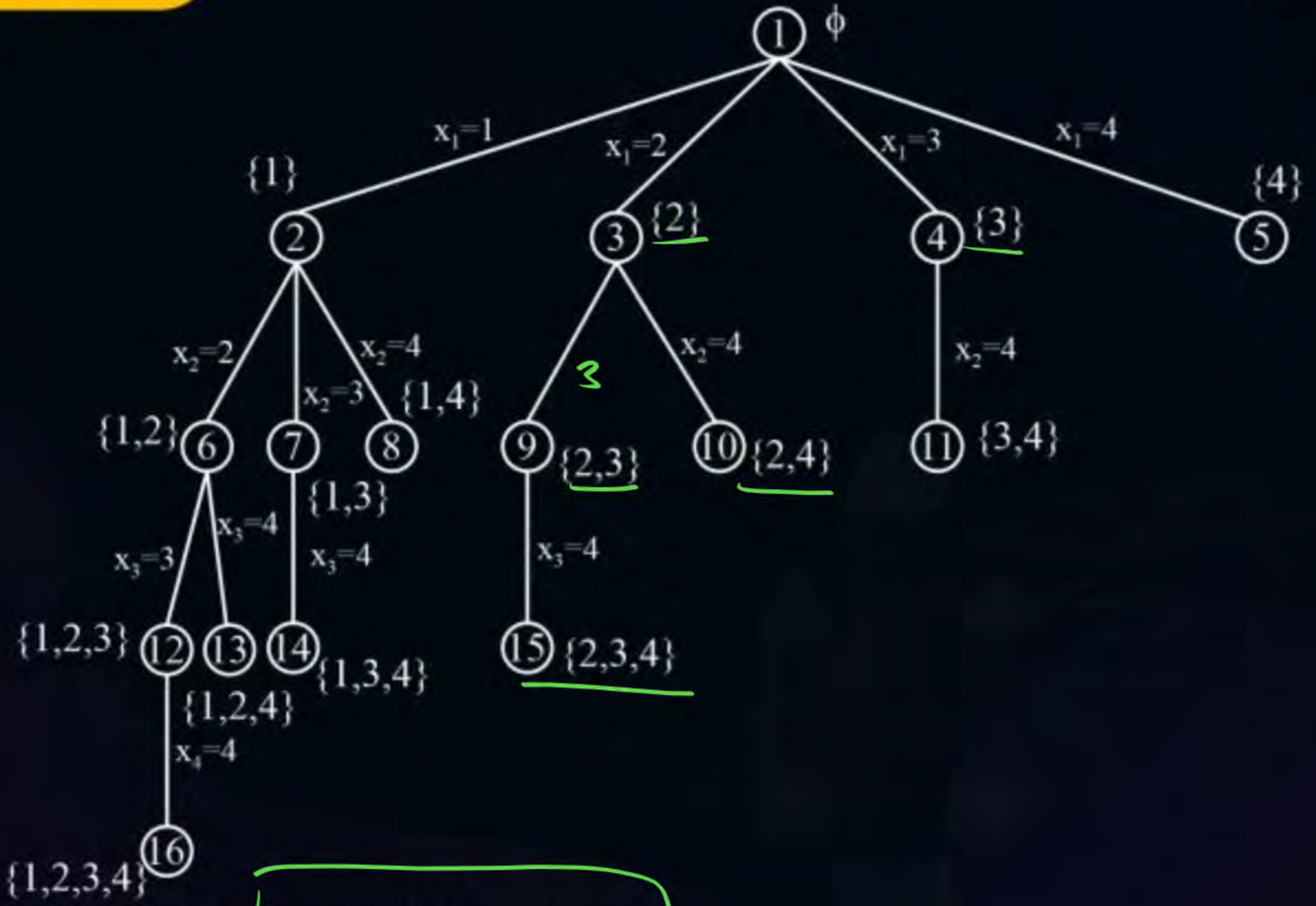
= Number of Subsets

$= 2^n$



Topic : Back Tracking

State Space Tree for n = 4



In this tree, each node is representing 1 possible subset.



Topic : Back Tracking

$\{a_1, a_2, a_3\}$

ϕ

a_1

a_2

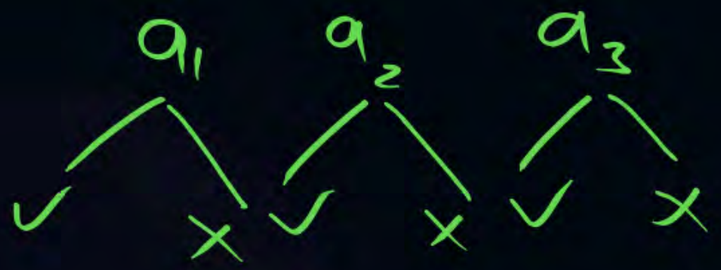
a_3

$a_1\ a_2$

$a_1\ a_3$

$a_2\ a_3$

$a_1\ a_2\ a_3$



Subsets = $2^3 = 8$



Topic : Back Tracking

For 4 elements, how many subsets?

$= 2^4 = 16$

And no. of nodes/states in Tree also 16.



Topic : Back Tracking

Test: Graph Coloring

Bonus

- You are given a graph having n vertices/nodes.
- You have to color this graph using minimum colors such that no two adjacent nodes have the same color.



Topic : Back Tracking

#Q. How many States gets generated for 1st ~~optional~~ optimal solution.

56.5%

(Bonus)

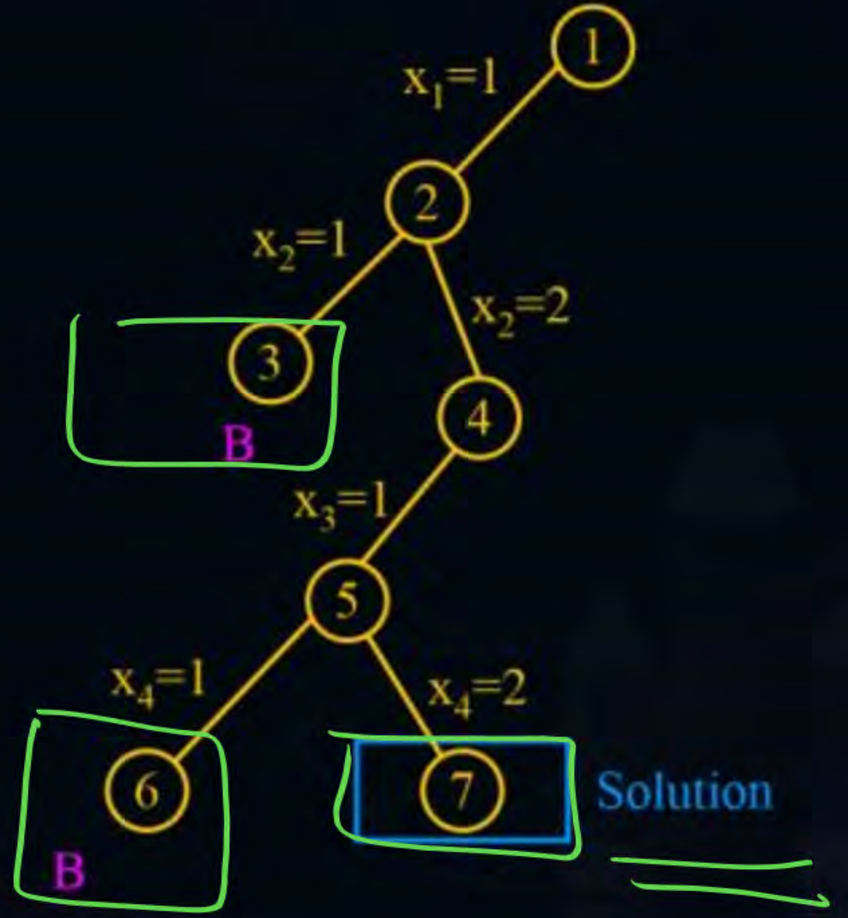




Topic : Back Tracking









Topic : Back Tracking

Branch and Bound

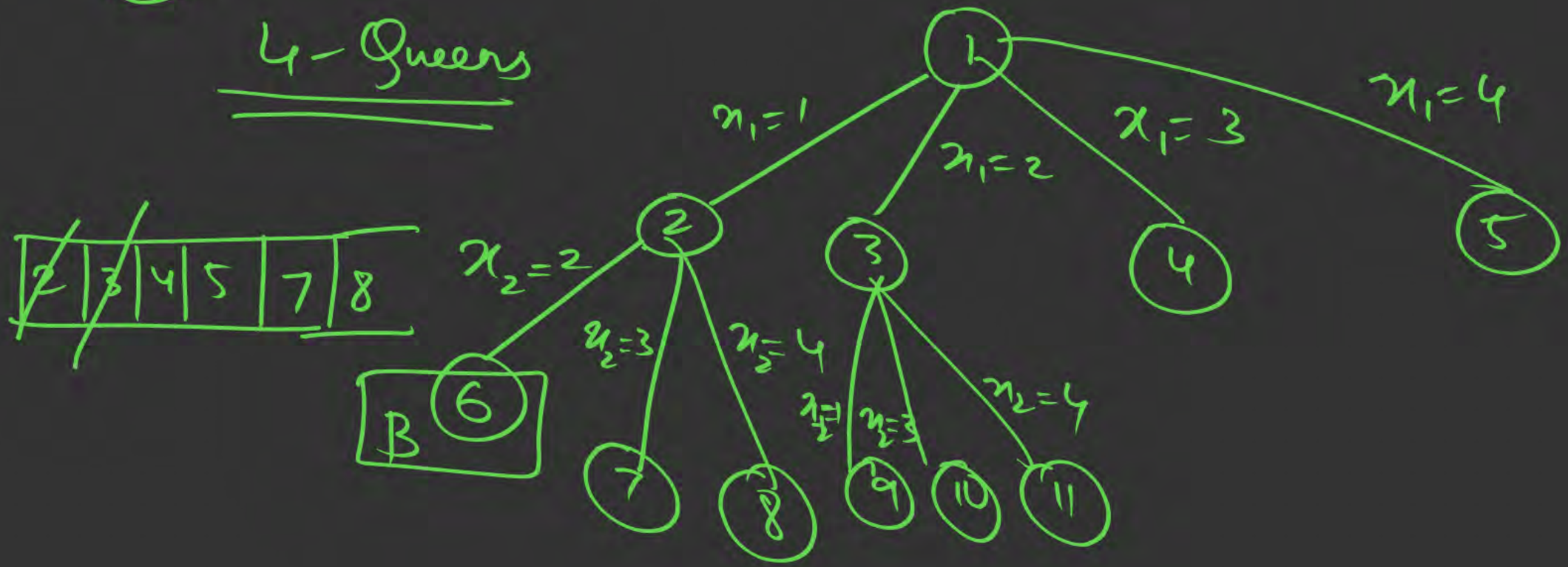
- Mostly same idea as that of Backtracking but it uses different Searching approach.
- Backtracking → DFS + Bounding Function
- Branch & Bound → BFS + Bounding Function

DFS → Stack LIFO

BFS → Queue FIFO

eg

4-Queens

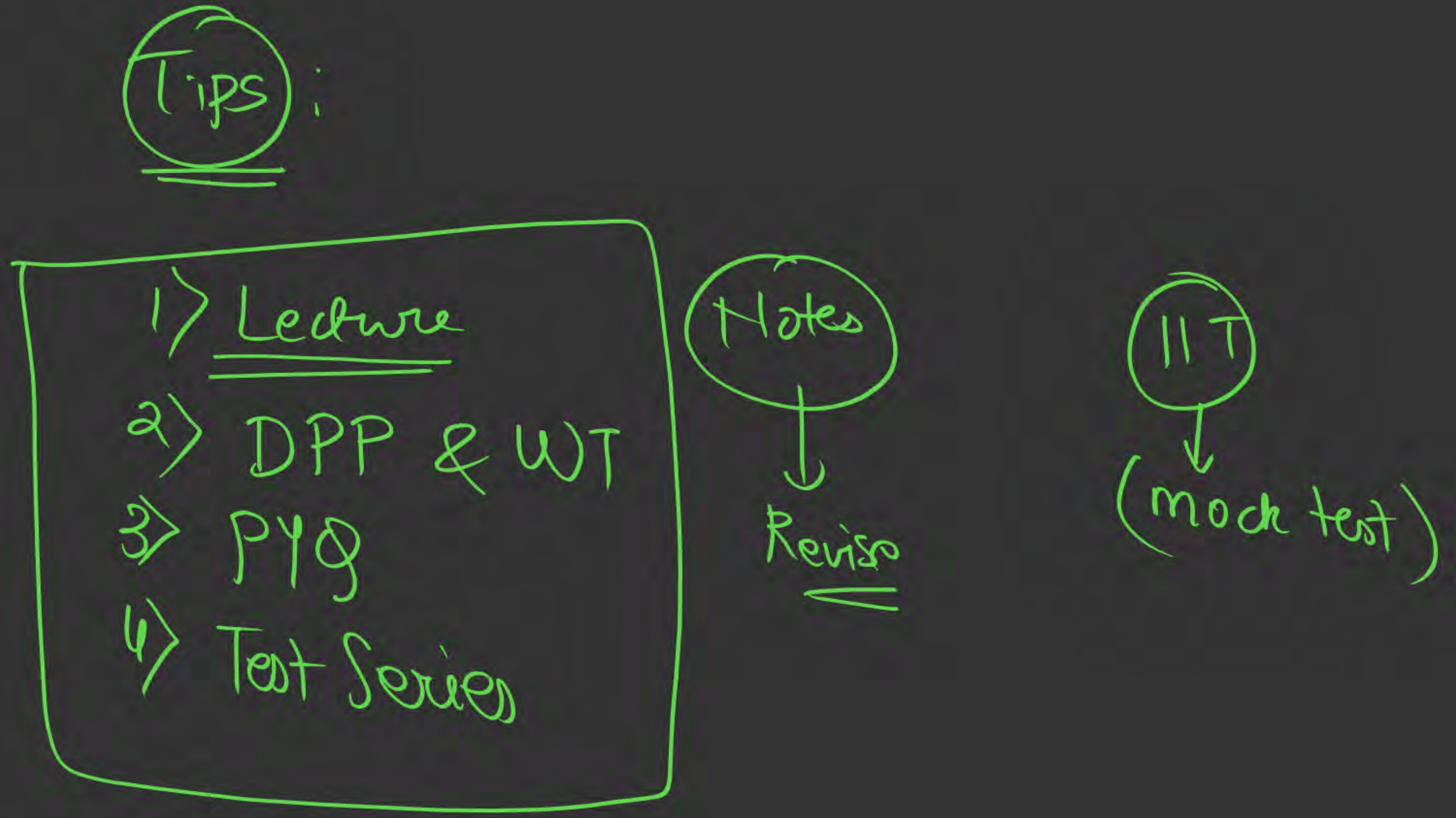


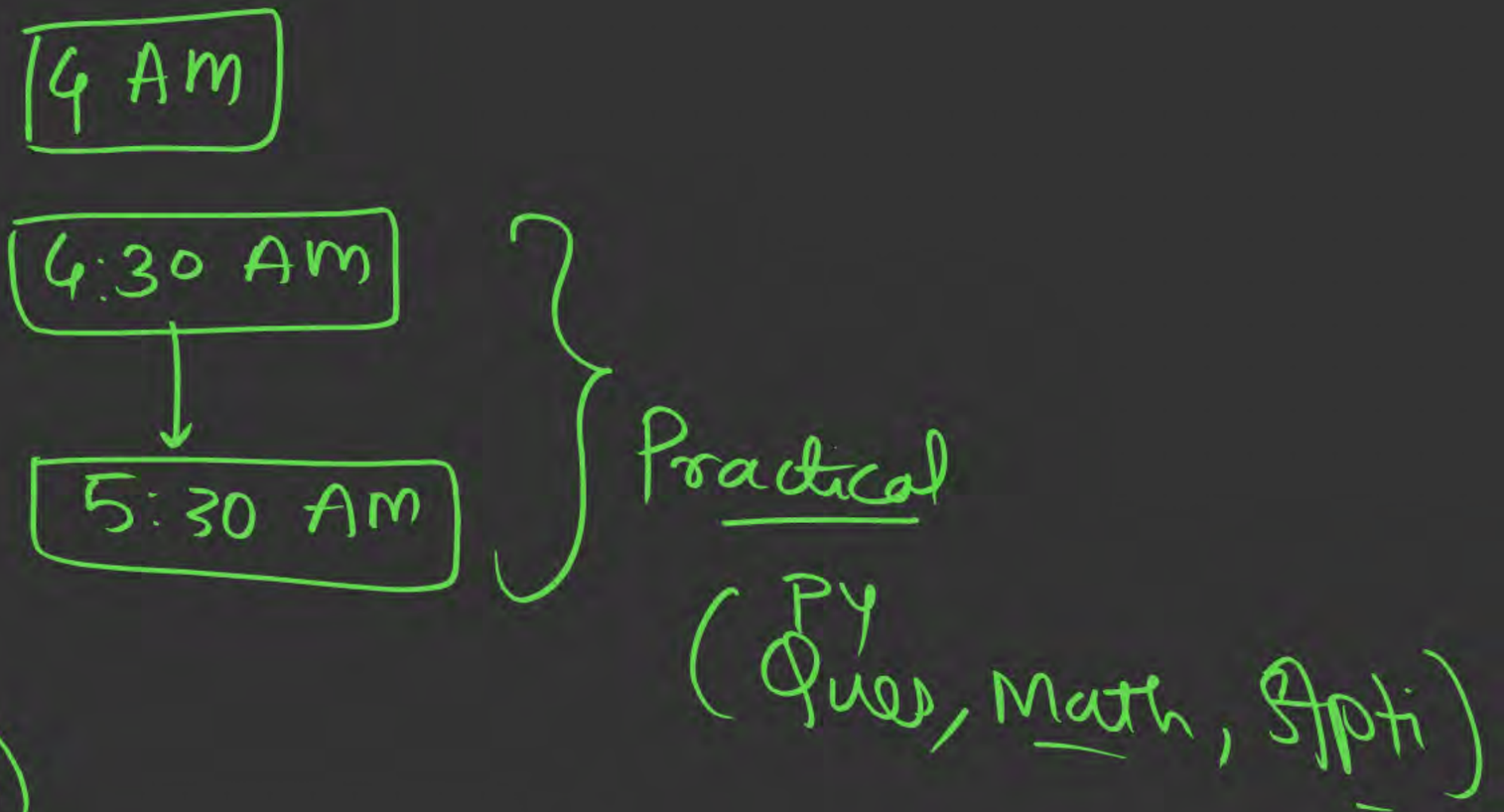
★ Back-tracking (Popular)

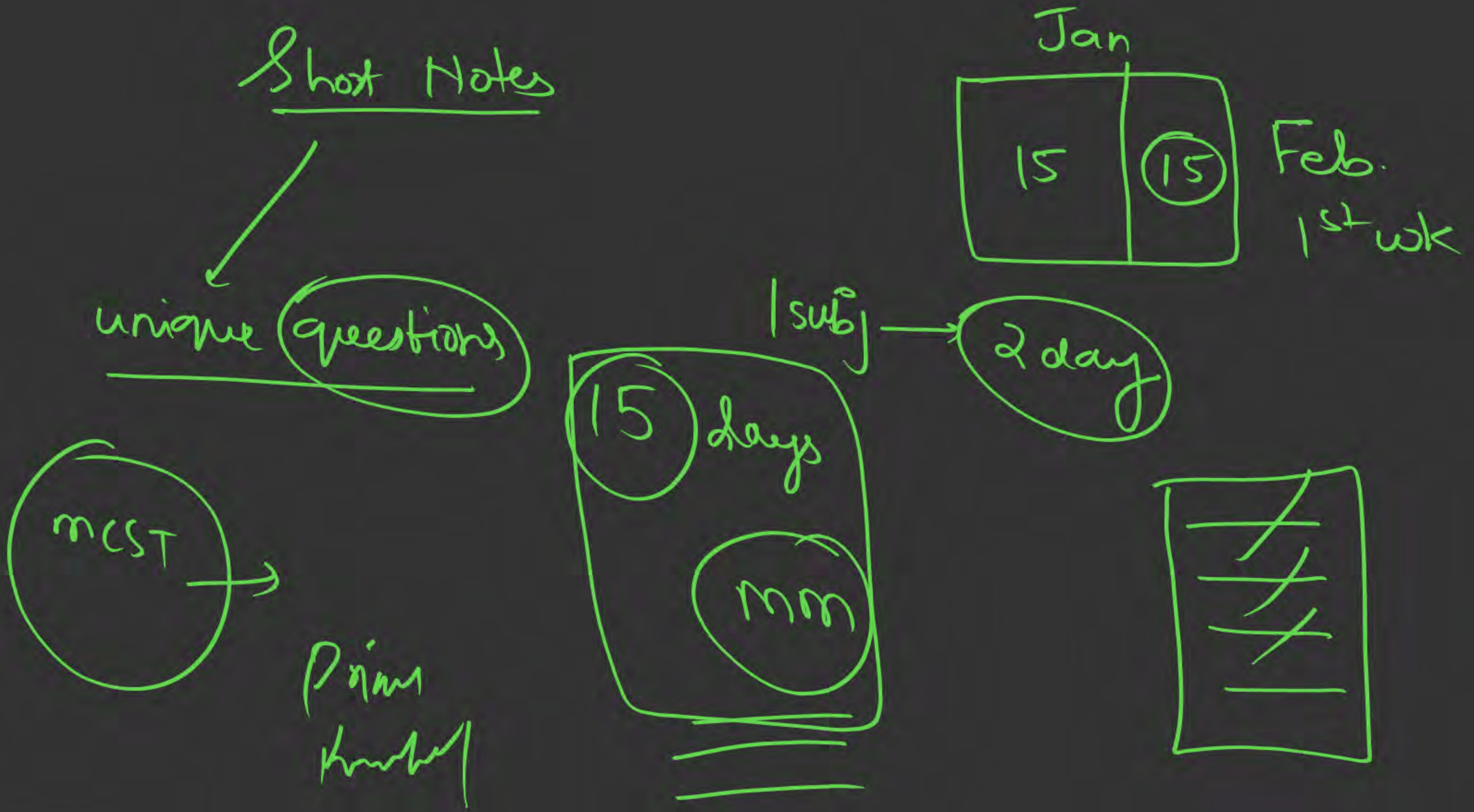
1) n-Queens
2) Graph Coloring
3) Sum of Subsets.
4) 0/1 knapsack
5) Hamiltonian Cycle.

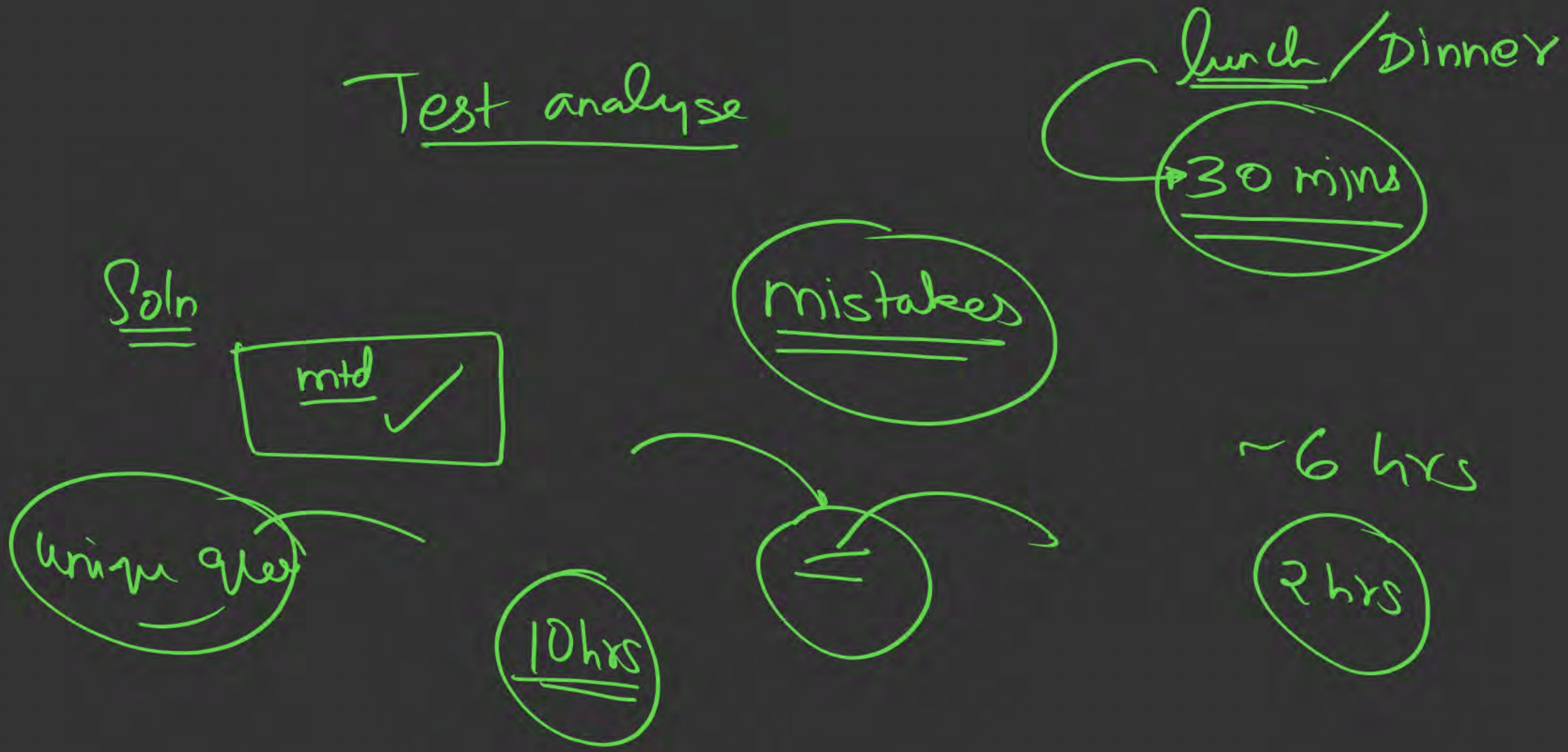
Branch-Bound: Popular Problems

1) n-Queens
2) TSP
3) Job Seq with Deadline
4) 0/1 Knapsack


Tips :
1) Lecture
2) DPP & WT
3) PYQ
4) Test Series
Notes
Revise
IIT
(mock test)


4 AM
4:30 AM
5:30 AM
Practical
(PY
Ques, Math, Sfpti)
1-1 wk
Nov-end
(Full length)


Short Notes
Jan
15
15
Feb.
1st wk
unique questions
1 subj
2 day
15 days
mm
MCST
Prime


Test analyse
lunch / Dinner
30 mins
Soln
mistakes
mtd
~6 hrs
10hrs
2 hrs


June
Nov
Dec
Jan
Feb
Toast
dg



Summary

- OBST → (part 2)
- Backtracking
- Branch Bound



THANK - YOU